~~Expiriment 8 UART & Serial Device Interface~~ ART
Expiriment 9 EEPROM 10/20/26

Purpose
Investigate the PIC16F883 data EEPROM: read & write stored data, Verify retention across power cycles, measure operation timing, & apply reliable nonvolitile storage in embedded systems.
Repository Setup: Create one Git repository for Lab 9. Add the student lab .gitignore before staging project files. Keep the complete development projects & follow the Lab Standards repository rules.

Standards & References
- RCET 3375 Lab standard
- RCET PIC-AS style Guide
- PIC16F883 data sheet & Family manual
- DLG7137 display datasheet
- Lab2 display circuit

Part 1 - EEPROM Persistance Proof of Life

Goal
Select A or B in RAM, deliberately save it to data EEPROM address 0x00, & Continuously display the stored EEPROM value. Measure read/write timing & verify retention after a power cycle.

Required Program behavior
- Buttons: Use two active-Low buttons with internal weak pull-ups: The Select button (RB0) & write button (RB1). Debounce both Pressing & releasing each button in software.
- Normal display: When the write button is ~~held~~ released, continuously read EEPROM address 0x00 in the main Loop & display the stored A, B, or ? for other values.
- Choose a Value: While the write button is held, display the pending RAM selection, initially A. Each valid Select button press toggles the selection between A & B. Pressing the select button while the write button is released leaves the Selection unchanged.
- Save a Value: A valid write button press enters selection mode. On its debounced release, Save the pending RAM value to EEPROM Once, wait for the operation to finish, & resume displaying the stored value.
- Write discipline: An EEPROM write may start only after a complete, debounced write button press-&-release cycle. Do Not write at startup, during normal main-loop reading, when the select button is pressed, or repeatedly while the write button is held. Contact bounce on either button must not produce an extra write.
- Timing diagnostics: RA1 is HIGH only during each EEPROM read, & RA2 is HIGH only during the EEPROM write operation, including the wait for completion. Keep both Low otherwise. Exclude display-Update time from the read diagnostic & preserve the uninterrupted EEPROM write-unlock sequence.

Experiment 9 EEPROM

ART
10/20/26

Before Lab

Prepare:

(pg 301)
- the PIC16F883 EEPROM SFR maps & read/write procedure, including the protected write sequence, write-complete indication, & interrupt precautions
- the circuit schematic & electrical-loading checks for both buttons, display, & diagnostics;
- flowcharts for the main loop, select button handling, write button handling, software debouncing, EEPROM read/write, & display update;
- Predicted display & diagnostic behavior for unrecognized EEPROM data, selection without saving, saving on write button release contact bounce, & power cycling.

EEPROM durability: From Section 17.5, document the minimum & typical byte endurance for both temperature ranges (D120/D120A), typical & maximum write time (D122), characteristics data retention (D123), & total EEPROM writes before refresh (D124). Explain endurance, retention, & refresh using Section 10.3.1.

Endurance calculation: Suppose a programming error repeatedly writes the same EEPROM address, starting another write as soon as the previous one finishes. Estimate the time to consume the minimum specified byte-endurance budget using the typical & maximum write times for both temperature ranges.
The calculation is: $T_{budget} = N_{writes} \cdot t_{write}$

Worked example ($-40°C$ to $+85°C$, typical write time):
Multiply the minimum byte endurance by the typical time per write to find the elapsed time, then convert seconds to minutes.

$$T_{budget} = 100{,}000 \text{ writes} \cdot 5 \frac{ms}{write} = 500{,}000 ms = 8 min\ 20 s$$

Complete the remaining three combinations using the maximum write time & the extended-temperature endurance value, show all substitutions, arithmetic, conversions, & units. State which software overhead you ignored, & explain why the endurance specification is a planning limit rather than an exact failure point. A mistaken EEPROM write inside the main loop could use up the specified endurance budget in minutes & permanently damage that address. Verify that your code cannot write repeatedly before leaving it running. Do not perform a destructive endurance test.

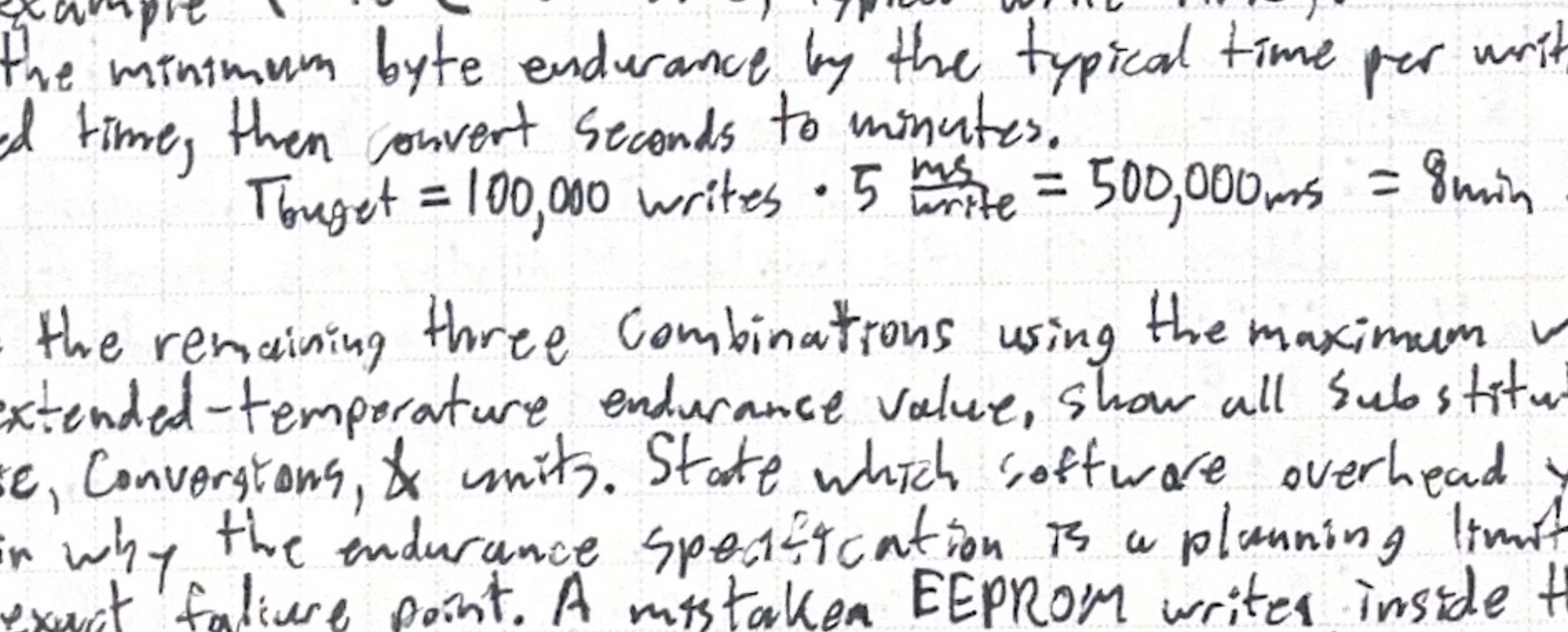


Figure 1-1 2 push buttons & display

ART
10/20/26

Experiment 9 EEPROM

ART
10/20/26

```
Main:
    CALL ReadEEPROM             ; Read Data in EEPROM address 0x00
    MOVWF DisplayRegister       ; Data read is in W, send to display
    MOVWF PORTC

    StartUpSequence:
    MOVF DisplayRegister,W
    XORLW 0x41                  ; Test current display for A
    BTFSS STATUS,2              ; Test the Zero bit to see if A matched
    goto TestB
    goto CheckButtons

    TestB:
    MOVF DisplayRegister,W
    XORLW 0x42
    BTFSS STATUS,2              ; Test the Zero bit to see if B matched
    goto StartupDisplay

    CheckButtons:
    CALL CheckWritePressed      ; See if the write button is pressed
    goto Main

    StartupDisplay:
    MOVLW 0x41
    MOVWF DisplayRegister
    goto CheckButtons
```

```
;ReadEEPROM reads data in address 0x00 and puts it in W
;Diagnostic port is toggled to show this on RA1
ReadEEPROM:
    BSF PORTA,1                 ; Data is being read Bit ON
    BANKSEL EEADR               ;
    CLRF EEADR                  ; Read Data in address 0x00

    BANKSEL EECON1              ;
    BCF EECON1,7                ; Read data in EEPROM not program Memory
    BSF EECON1,0                ; Begin reading
    BANKSEL EEDATA              ;
    MOVF EEDATA,W               ; Get data from EEDATA register
    BANKSEL PORTA               ;
    BCF PORTA,1                 ; Data is no longer read Bit OFF
    RETURN

CheckWritePressed:
    BTFSC PORTB,1               ; Test if button has been pressed
    RETURN                      ; If it hasent, RETURN

    CALL Delay10ms              ; Wait 10 milliseconds
    BTFSC PORTB,1               ; If still pressed continue
    RETURN                      ; If not, Bounce Detected so RETURN

    WriteHeld:
    BTFSC PORTB,1               ; Test if button has been Released
    CALL Delay10ms              ; If it has been, wait 10ms
    BTFSC PORTB,1               ; Test again for debounce
    goto EEPROMWrite            ; If it has goto EEPROMWrite

    BTFSC PORTB,0               ; Test Select button
    goto WriteHeld              ; If not pressed Stay here until Write is released
    CALL Delay10ms              ; Test for bounces
    BTFSC PORTB,0
    goto WriteHeld              ; Bounce detected, stay here

    SelectPressed:
    BTFSC PORTB,0               ; Test if Select button has been releasted
    CALL Delay10ms              ;
    BTFSS PORTB,0               ; Check for bouncing
    goto SelectPressed          ;

    MOVLW 0x03
    XORWF DisplayRegister,F     ; Switch the pick: 'A' becomes 'B', 'B' becomes 'A
    MOVF DisplayRegister,W      ;
    MOVWF PORTC                 ; Display new letter on PortC
    goto WriteHeld
```

```
EEPROMWrite:
BANKSEL PORTA
BSF PORTA,2                     ;write marker ON
BANKSEL EEADR
CLRF EEADR                      ;the address to write is 0
MOVF DisplayRegister,W
MOVWF EEDATA                    ;the letter to store
BANKSEL EECON1
BCF EECON1,7                    ;pick data EEPROM
BSF EECON1,2                    ;allow writing
BCF INTCON,7                    ;make sure no interrupt can get in the way of the next lines
MOVLW 0x55                      ;the chip needs this exact code, in this exact order,
MOVWF EECON2                    ;with nothing in between, or it ignores the write
MOVLW 0xAA
MOVWF EECON2
BSF EECON1,1                    ;start the write

BTFSC EECON1,1                  ;the chip clears this bit when it is finished
goto $-1                        ;if it is still writing then wait
BCF EECON1,2                    ;turn writing off again so nothing can write by accident
BANKSEL PIR2
BCF PIR2,4                      ;clear the "write finished" flag
BANKSEL PORTA
BCF PORTA,2                     ;write marker OFF
RETURN
```

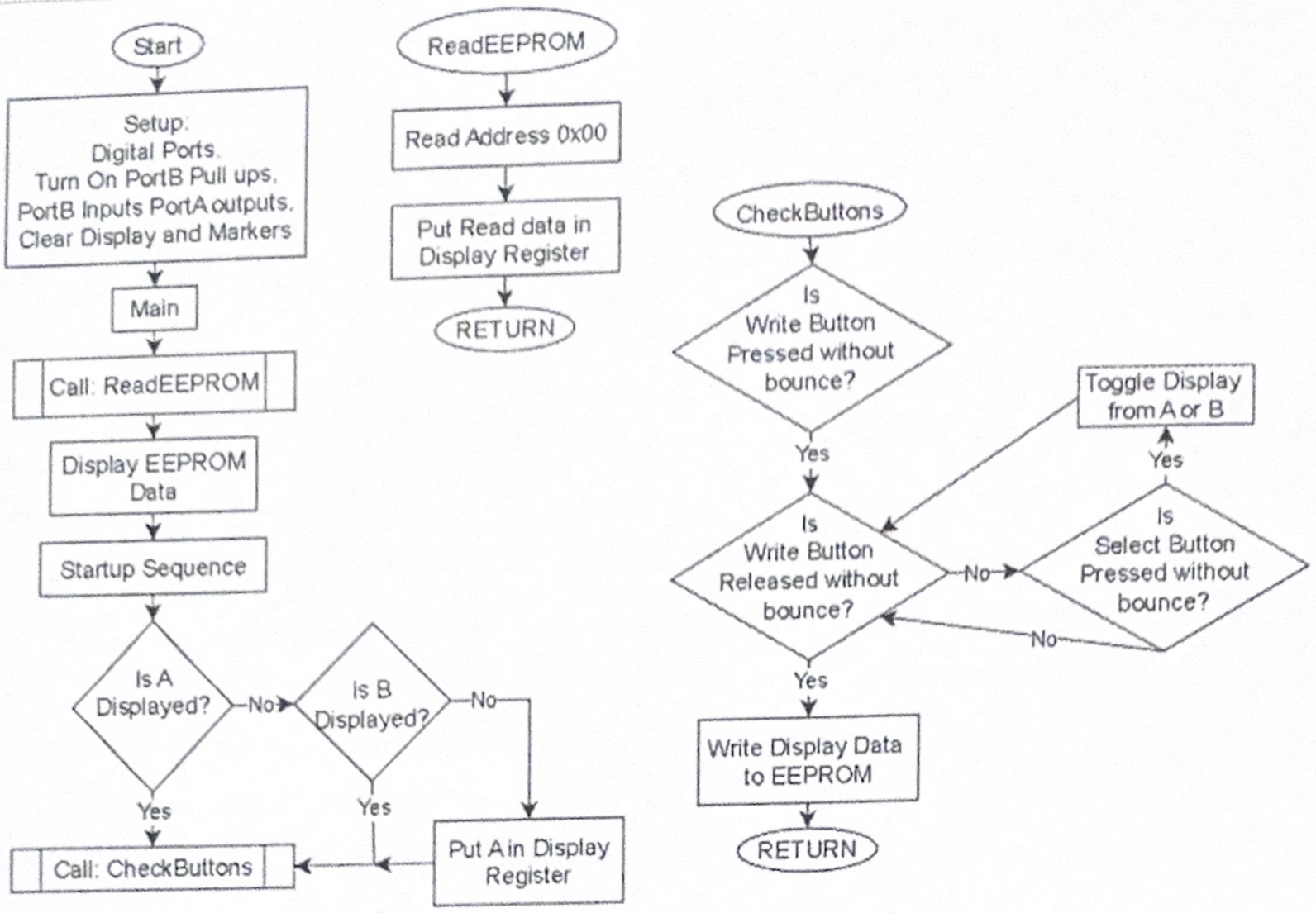
Code 1-1 EEPROM Single letter Save with debounced buttons

*The StartupSequence Loads the EEPROM with a 0x41 only once so it contains the proper value in 0x00 to toggle.

ART
10/21/26

Experiment 9 EEPROM

ART
10/21/26



FC 1-1 for Code 1-1

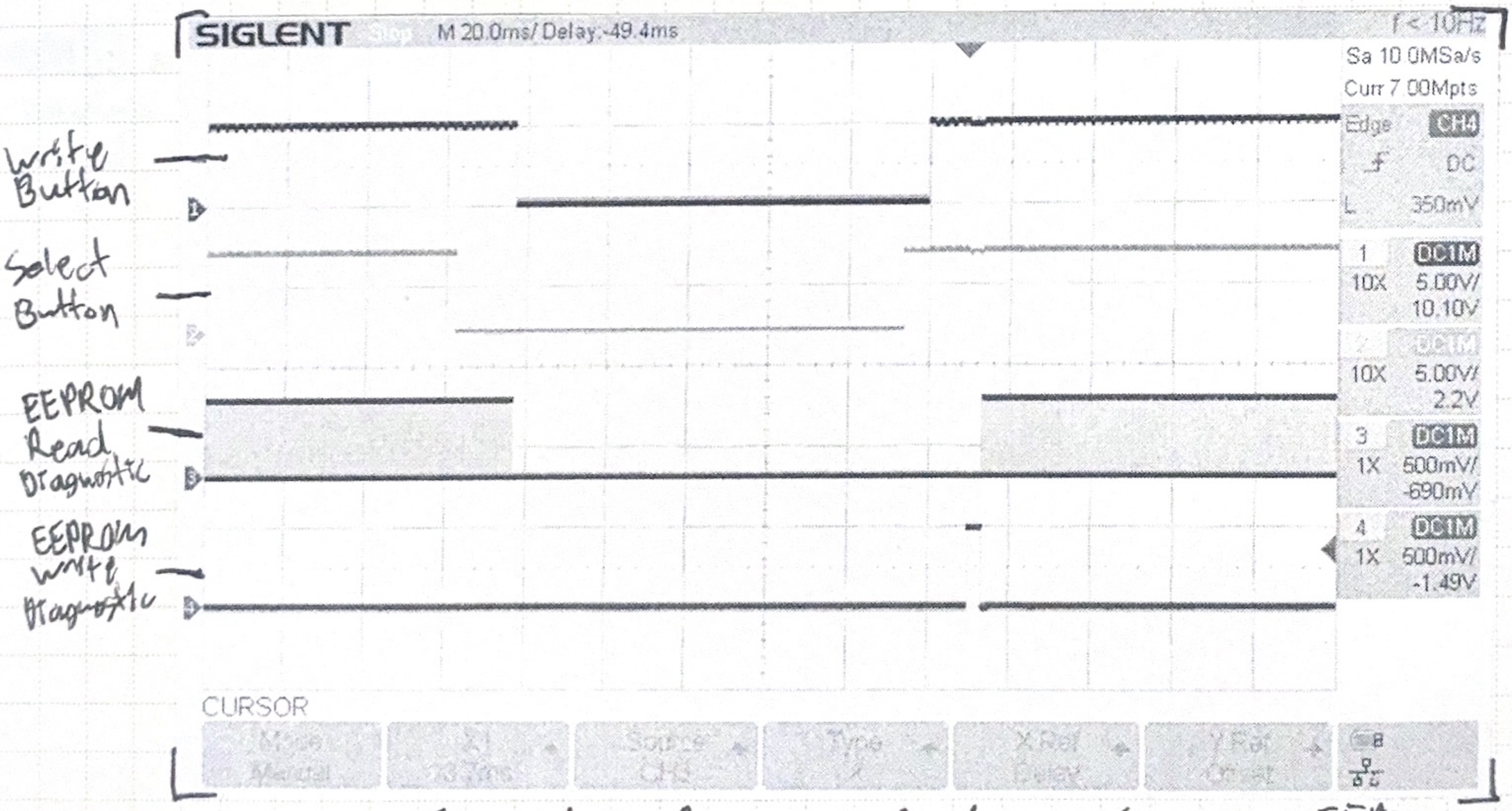


Figure 1-2 Capture of diagnostics on EEPROM

JR
10-21-26

ART
10/21/26

Expiriment 9 EEPROM

ART
10/21/26

Part 2 - Reusable EEPROM Read/Write Routines & Timed Selection

Goal

Extend Part1 with reusable EEPROM routines, a nonblocking write, & a separate verification routine. Replace hold-to-select with a timed selection mode.

I/O assignments Table 2-1

Pin	Direction	Function
RA0	Output	Selection-mode LED, HIGH during selection
RA1	Output	Main-loop diagnostic pulse, once per iteration
RA2	Output	EEPROM error LED, HIGH when `EEPROM_ERROR` is set
RB0	Input	Select button, active LOW with internal pull-up
RB1	Input	Write button, active LOW with internal pull-up

Keep PortC connection from part1

EEPROM Routines

GPR	Purpose
ee_address	EEPROM byte address for read or write
ee_data	Byte returned by a read or written during a write
ee_status	`EEPROM_READ` , `EEPROM_WRITE` , and `EEPROM_ERROR` status flags

Use three RAM general purpose registers (GPR's). Define the three status flag bit positions with EQU.

The EEPROM subroutines must use only these three RAM GPR's & work without modification in other programs. The calling application may use additional RAM GPR's.

YOU May choose whether to use polling or the EEPROM write-Complete interrupt to determine when a write finishes. If using the interrupt, the ISR should call EEPROMVerifyWrite Once per completed write. You may use internal helper subroutines for EEPROM read/write, but call them only from the EEPROM routines & document the maximum stack depth.

Use these routine labels for external calls:

- EEPROMRead: Load ee_address before the call. Set the EEPROM_Read status flag on entry, Place the EEPROM byte in ee_data, & clear the EEPROM_Read Status flag before returning.
- EEPROMWrite: Load ee_address & ee_data before the call. If EEPROM_Write or EEIF is set, Return immediatly without changing ee_status. Otherwise, Compare the stored byte at ee_address with the requested value in ee_data without overwriting ee_data. If the values match, return without waiting. Preserve any existing EEPROM_ERROR status flag. Otherwise, set the EEPROM_Write status flag, start the protected write Sequence, & return without waiting for the write to finish. Keep the unlock sequence uninterrupted & follow the interrupt precautions from part 1. Before returning, restore GIE to its previous state, enabling interrupts only if they were enabled on entry. You may use a bit in ee_status to Save the previous GIE state; Clear that bit after restoring GIE.
- EEPROMVerifyWrite: Call after the hardware write finishes. Compare the byte at the ee_address with the expected value still in ee_data. Do Not overwrite eedata during the comparison. If they match, clear ee_status. If the differ, clear the EEPROM_Write status flag & set the EEPROM_ERROR status flag. Clear EEIF & WREN before returning.

Neither EEPROMwrite nor EEPROMVerifyWrite should automatically retry a write on failure.

ART
10/22/26

Experiment 9 EEPROM

ART 10/22/26

Button Service

- Service the select & write buttons by polling or interrupts. Keep the part 1 button debouncing & accept one action per press.
- Do not start another EEPROM operation or change ee_address or ee_data while a write is awaiting verification
- Ignore button events while either the EEPROM_READ or EEPROM_WRITE status flag are set. The EEPROM_ERROR status flag does not block the buttons.

Timed Selection

- Normal operation: RA0 is off. Display the character stored at EEPROM add. 0x00
- First Select button (RB0) Press: Enter selection mode, select A, display A, turn RA0 ON, & start an approximately one-second timeout.
- Additional Select button Presses: Toggle the pending value between A & B, update the display, & restart the one-second timeout.
- Timeout: About one second after the last Select button press, exit selection mode, turn RA0 off, & resume the EEPROM stored-value display. A timeout does not start an EEPROM write.
- Write Button (RB1): A debounced press before the timeout exits selection mode, stops the selection timer, turns RA0 off, & calls EEPROM_write to save the pending character to address 0x00. Ignore write button presses outside selection mode.

Optional address-testing tip: In the application, define current_address with EQU as a RAM GPR & initialize it to 0X00. Use its value when loading ee_address for display reads & saved writes. Changing this one value makes it easier to test other EEPROM addresses. This is optional & is not part of the reusable EEPROM routines.

Main Loop behavior

Repeat these steps in Main:

1. Pulse RA1: Set & immediately clear the mainLoop diagnostic pin
2. Check write completion: If polling, check EEIF while the EEPROM_write status flag is set & call EEPROM_verify_write when the write finishes. If using the EEPROM write-complete interrupt, the ISR calls EEPROM_verify_write instead.
3. Update RA2: Set the error LED to match the EEPROM_ERROR status flag on every iteration.
4. Service timeout, then buttons: check the selection timeout before accepting button presses. Poll the buttons or process button events supplied by interrupts.
5. Update the DLG7137 display: Show the pending character while in selection mode. Otherwise, read EEPROM address 0x00 & display A, B, or ? for any other stored value. While the EEPROM_write status flag is set, keep displaying the last character without reading EEPROM

b. Repeat

Before Lab

Document the three GPR assignments, status flag bit positions, inputs & outputs of each EEPROM routine, & how GIE is saved & restored during the protected write sequence. Prepare a state diagram for selection, timeout, writing, verification, & errors. Include the one-second timing plan & an address/value test table. Document the RA0 & RA2 LED connections & loading, & the RA1 diagnostic pin. Explain why EEPROM reads do not consume write endurance & why checking for an unchanged value reduces wear.

ART 10/22/26

Experiment 9 EEPROM

ART 10/22/26

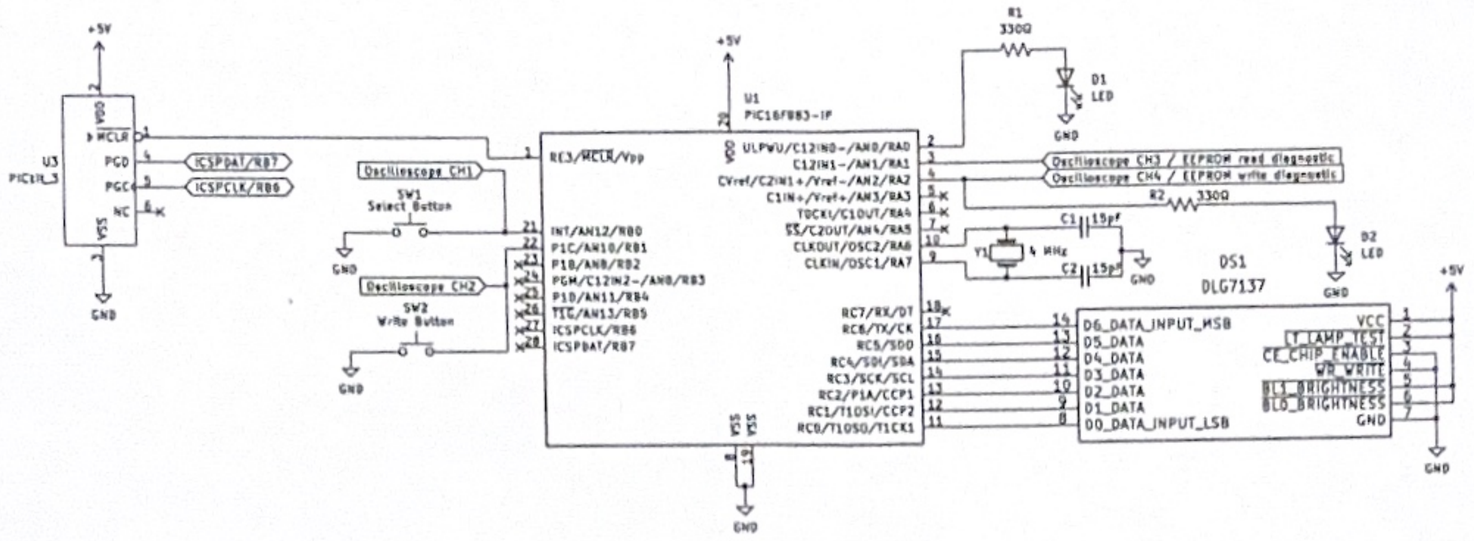


Figure 2-1 Connected LEDS to EEPROM Circuit timer

Code 2-1

```
; The error LED always matches the error flag
ErrorLed:
    BTFSC ee_status,EEPROM_ERROR    ;is there an error?
    goto ErrorOn
    BCF PORTA,2                     ;no error, LED OFF
    goto CheckTimeout
ErrorOn:
    BSF PORTA,2                     ;error, LED ON

; Every 10 ms count down the selection timeout
CheckTimeout:
    BTFSS PIR1,1                    ;has 10 ms gone by?
    goto CheckButtons               ;no
    BCF PIR1,1                      ;clear the flag for the next 10 ms
    BTFSS Selecting,0               ;only count while picking a letter
    goto CheckButtons
    DECFSZ TimeoutCount,F           ;one tick used up, is it zero?
    goto CheckButtons               ;not yet
    CALL ExitSelect                 ;time is up, leave selection (no write)

; Ignore the buttons while a read or write is happening
CheckButtons:
    MOVLW EE_BUSY
    ANDWF ee_status,W               ;W = the READ and WRITE bits
    BTFSS STATUS,2                  ;zero means not busy
    goto ShowLetter                 ;busy, so skip the buttons

; Select button
    BTFSS PORTB,0                   ;is it pressed? LOW = pressed
    goto SelectPressed              ;yes
    BTFSS SelectDown,0              ;it is released, did we count a press before?
    goto CheckWrite                 ;no, nothing to do
    CALL Wait10ms                   ;wait 10 ms to make sure it is really released
    BTFSC PORTB,0                   ;pressed again? skip the next line if yes
    CLRF SelectDown                 ;really released, so get ready for the next press
    goto CheckWrite
```

Checks for errors then moves to button press handling. If EEPROM is already Reading/writing it will not check buttons & just display the current letter.

```
;EEPROMWrite: in ee_address and ee_data.
;It starts the write and returns. It does NOT wait.
;It does nothing if a write is already going, or if the stored byte
;is already the same this saves wear on the EEPROM.
EEPROMWrite:
    BTFSC ee_status,EEPROM_WRITE    ;is a write already going?
    RETURN                          ;yes, leave everything alone
    BANKSEL PIR2
    BTFSC PIR2,4                    ;is EEIF still set from an old write?
    RETURN                          ;yes, leave everything alone
    CALL EEReadByte                 ;W = the byte stored right now
    XORWF ee_data,W                 ;zero if it is the same as what we want to save
    BTFSC STATUS,2
    RETURN                          ;same value, so do not write

    BSF ee_status,EEPROM_WRITE      ;a real write is starting
    BANKSEL EEADR
    MOVF ee_address,W
    MOVWF EEADR                     ;the address to write
    MOVF ee_data,W
    MOVWF EEDATA                    ;the byte to write


    ;MOVLW   1                      ;TEST ONLY: remove the ; on these two lines to write
    ;ADDWF   EEDATA,F               ;the WRONG byte, so the check fails on purpose


    BANKSEL EECON1
    BCF EECON1,7                    ;pick data EEPROM
    BSF EECON1,2                    ;allow writing

    BCF ee_status,EE_GIE_SAVED      ;start by assuming interrupts were OFF
    BTFSC INTCON,7                  ;were they ON?
    BSF ee_status,EE_GIE_SAVED      ;yes, remember that
GieOff:
    BCF INTCON,7                    ;turn interrupts OFF

    MOVLW 0x55                      ;the chip needs this exact code, in this exact order,
    MOVWF EECON2                    ;with nothing in between, or it ignores the write
    MOVLW 0xAA
    MOVWF EECON2
    BSF EECON1,1                    ;start the write (we do not wait)

    BTFSC ee_status,EE_GIE_SAVED    ;were interrupts ON when we came in?
    BSF INTCON,7                    ;yes, turn them back ON
    BCF ee_status,EE_GIE_SAVED      ;we are done with that memory bit
    BANKSEL PORTA                   ;back to bank 0
    RETURN
```

EEPROM Write with GIE Handeling. Turns off any interrupts during a write using a status bit EQU.

ART 10/22/26

Experiment 9 EEPROM

Code 2-2

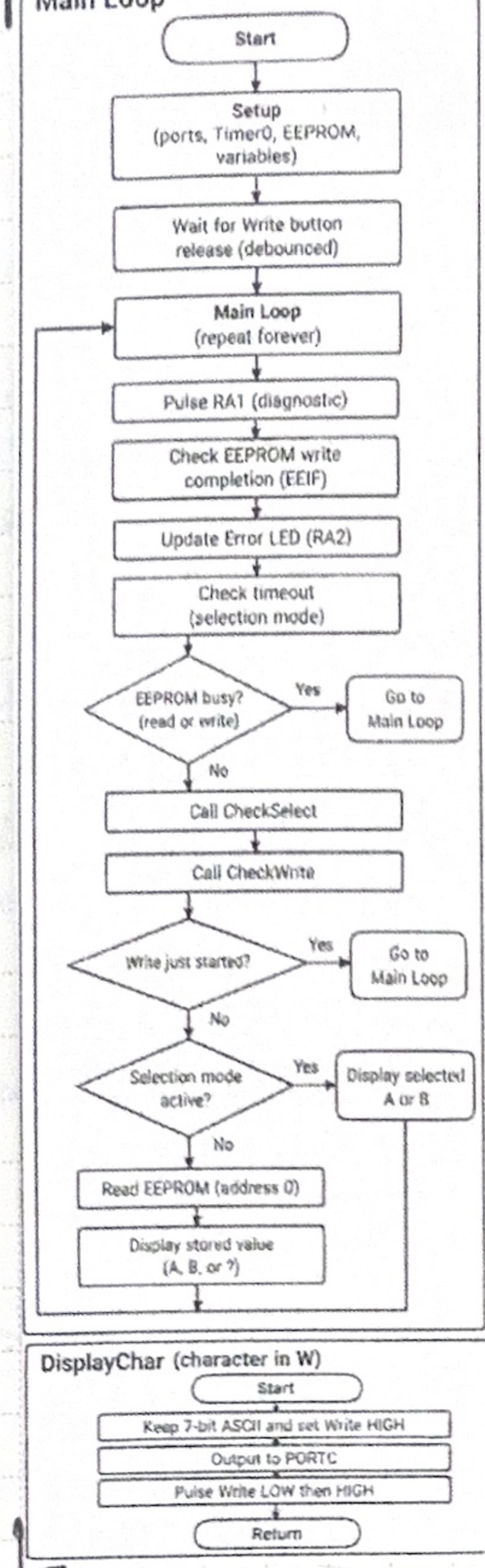
ART
10/22/26

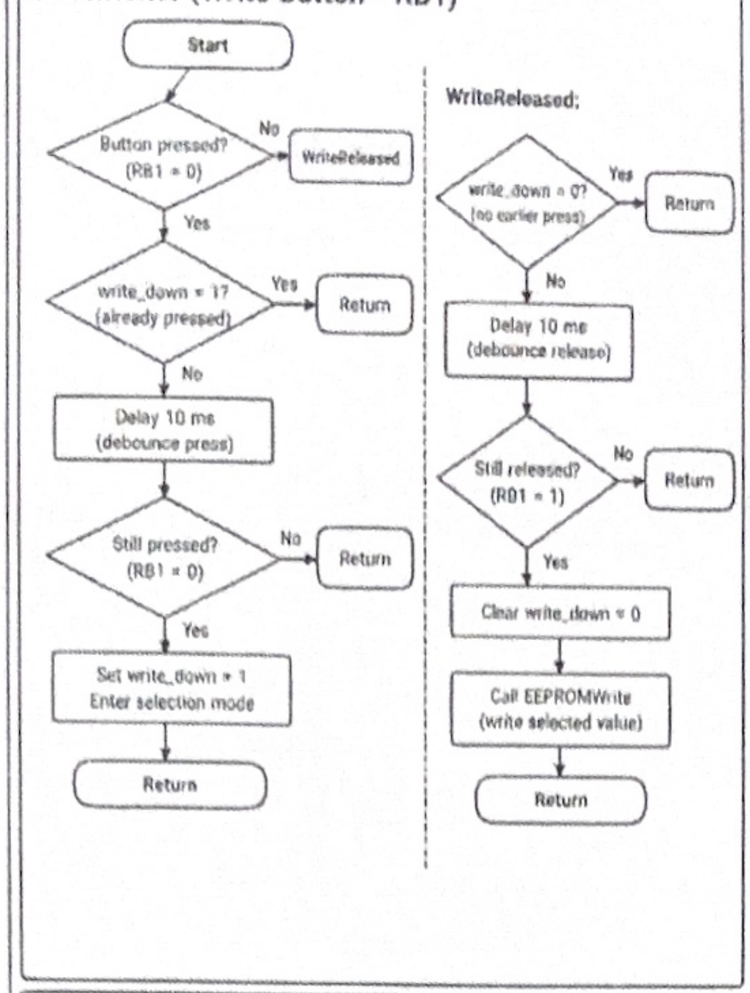
```
SelectPressed:
    BTFSC SelectDown,0          ;did we already count this press?
    goto CheckWrite             ;yes, ignore it
    CALL Wait10ms               ;wait 10 ms to make sure it is not bounce
    BTFSC PORTB,0               ;still pressed? skip the next line if yes
    goto CheckWrite             ;it was only bounce
    BSF SelectDown,0            ;remember we counted this press
    BTFSC Selecting,0           ;already picking a letter?
    goto TogglePick             ;yes, switch the letter
    BSF Selecting,0             ;no, this is the first press, so start picking
    MOVLW 0x41
    MOVWF Pending               ;start with 'A'
    BSF PORTA,0                 ;selection LED ON
    goto RestartTimeout
TogglePick:
    MOVLW 0x03
    XORWF Pending,F             ;'A' becomes 'B', 'B' becomes 'A'
RestartTimeout:
    MOVLW 100
    MOVWF TimeoutCount          ;a new full second (100 x 10 ms) starts now

;Write button
CheckWrite:
    BTFSS PORTB,1               ;is it pressed? (LOW = pressed)
    goto WritePressed           ;yes
    BTFSS WriteDown,0           ;it is released, did we count a press before?
    goto ShowLetter             ;no, nothing to do
    CALL Wait10ms               ;wait 10 ms to make sure it is really released
    BTFSC PORTB,1               ;pressed again? skip the next line if yes
    CLRF WriteDown              ;really released, so get ready for the next press
    goto ShowLetter
WritePressed:
    BTFSC WriteDown,0           ;did we already count this press?
    goto ShowLetter             ;yes, ignore it
    CALL Wait10ms               ;wait 10 ms to make sure it is not bounce
    BTFSC PORTB,1               ;still pressed? skip the next line if yes
    goto ShowLetter             ;it was only bounce
    BSF WriteDown,0             ;remember we counted this press
    BTFSS Selecting,0           ;Write only counts while picking a letter
    goto ShowLetter             ;not picking, so ignore it
    CALL ExitSelect             ;stop the timer and turn the LED OFF
    MOVF CurrentAddress,W
    MOVWF ee_address            ;where to save
    MOVF  Pending,W
    MOVWF ee_data               ;what to save
    CALL  EEPROMWrite           ;start the write (it does not wait)

;Update the display
ShowLetter:
    BTFSC Selecting,0           ;picking a letter?
    goto ShowPending            ;yes, show the pick
    BTFSC ee_status,EEPROM_WRITE ;is a write happening?
    goto Main                   ;yes, leave the display alone, do not read the EEPROM
    MOVF CurrentAddress,W
    MOVWF ee_address
    CALL EEPROMRead             ;the stored letter comes back in ee_data

    MOVF ee_data,W
    MOVWF Display
    XORLW 0x41                  ;is it A
    BTFSC STATUS,2
    goto ShowStored             ;if yes show it
    MOVF Display,W
    XORLW 0x42                  ;is it B
    BTFSC STATUS,2
    goto ShowStored             ;if yes show it
    MOVLW 0x3F                  ;it is neither, so show a question mark
    MOVWF Display
ShowStored:
    MOVF Display,W
    CALL DisplayLetter          ;put the letter on the display
    goto Main
ShowPending:
    MOVF Pending,W
    CALL DisplayLetter          ;show the letter we are picking
    goto Main

;ExitSelect: leave selection mode (used by the timeout and by Write).
ExitSelect:
    CLRF Selecting              ;not picking anymore
    CLRF TimeoutCount           ;stop the timer
    BCF PORTA,0                 ;selection LED OFF
    RETURN

;DisplayLetter: the letter to show is in W.
DisplayLetter:
    BANKSEL PORTC               ;make sure we are in bank 0 (W is not changed)
    MOVWF PORTC                 ;send the letter to the display
    RETURN
```

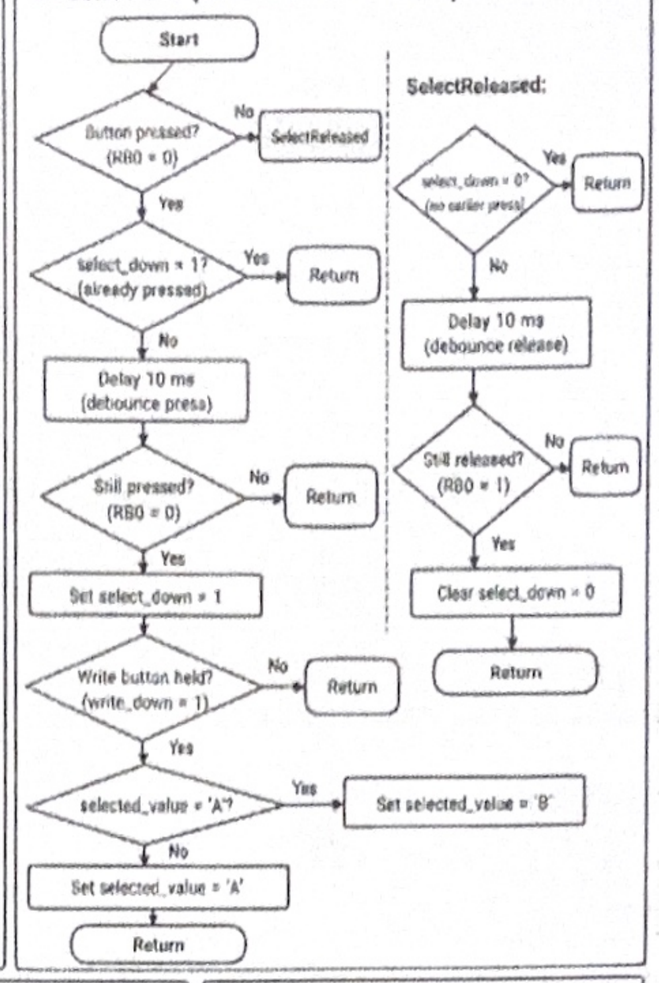
Experiment 9 EEPROM

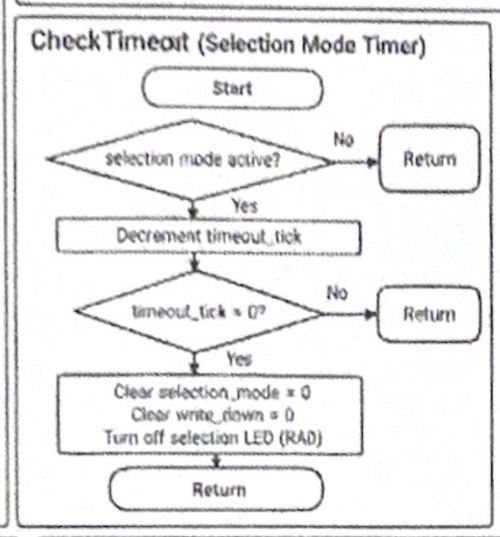
ART 10/22/26

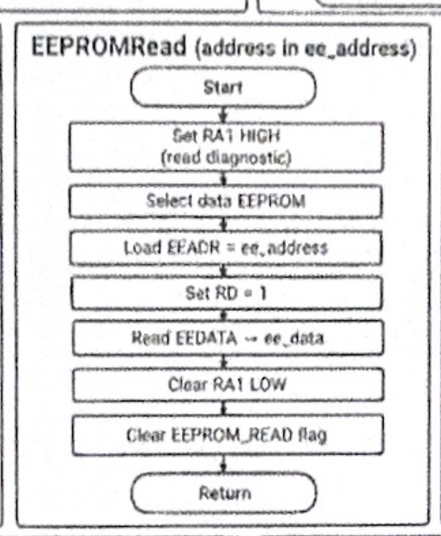
FC 2-1 for code Blocks 2-1 & 2-2

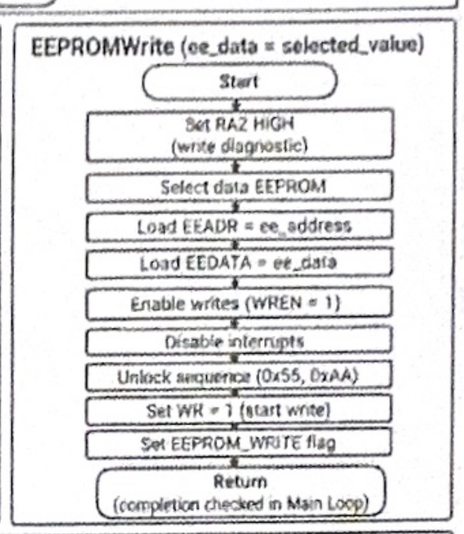












Important Variables (GPRs)

selected_value	-	pending RAM selection (A or B)
stored_value	-	value read from EEPROM
write_down	-	1 when Write button is held
select_down	-	1 when Select button is held
ee_address	-	EEPROM address (used by routines)
ee_data	-	data byte (used by routines)
ee_status	-	status bits (READ, WRITE, ERROR, GIE_SAVED)
selection_mode	-	1 when selecting A or B
timeout_tick	-	counts Timer0 overflows (selection timeout)

Constants and Status Bits

DATA_EE_ADDR	= 0x00	(EEPROM address)
TIMEOUT_OVERFLOWS	= 15	(≈ 1 second)

ee_status bit positions:

bit 0	EEPROM_READ	(1 while a read is in progress)
bit 1	EEPROM_WRITE	(1 while a write is in progress)
bit 2	EEPROM_ERROR	(set if write verification fails)
bit 3	GIE_SAVED	(saved global interrupt enable)

Timer0 (Selection Timeout)

- 4 MHz clock, prescaler 1:256
- 15 overflows ≈ 15 × 65.536 ms ≈ 1 second

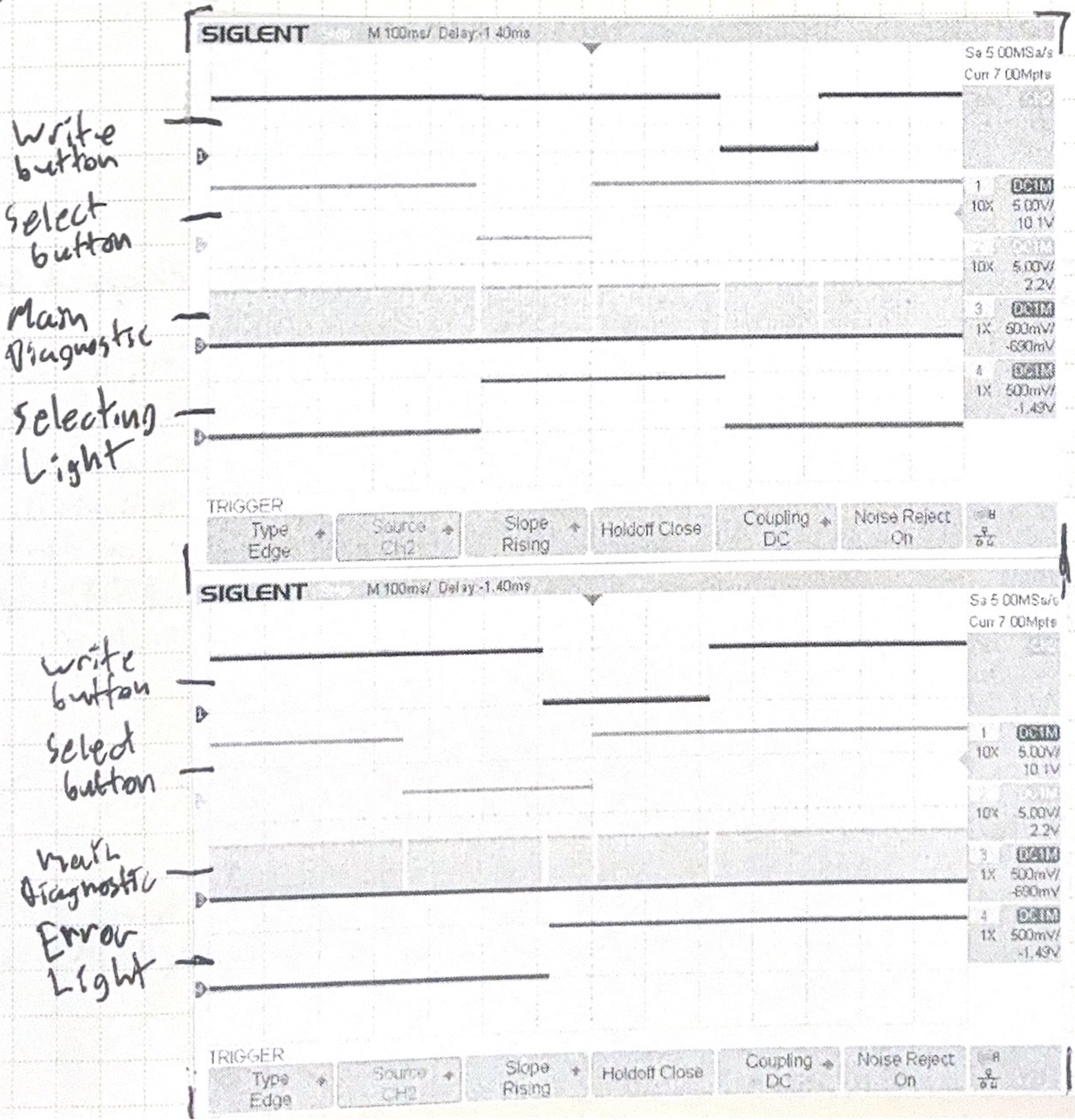


Figure 2-2 O-Scope capure from Figure 2-1 Selection & Write

Figure 2-3 o-scope capture from Figure 2-1 Error thrown

TJR 10-23-26 ART 10/22/26

Expiriment 9 EEPROM

Part 3 - Four-Digit Electronic Lock

Goal

Extend Part2 into a Keypad-Controlled electronic Lock. Enter a Four-digit PIN followed by # to activate a simulated Solenoid output for one Second, The PIN is read from data EEPROM.
Saftey: Do not connect a Solenoid directly to a PIC output. A real inductive Load requires an appropriate driver & transient protection. Keep the simulated output inactive during reset & startup as well as normal locked operation.

Initial Pin Setup

Before running Part3, manually program a four-digit PIN into four consecutive data EEPROM addresses, with one decimal digit (0-9) in each address. Choose the addresses & varify the stored digits. This is a one-time-setup. PIN changes are not part of Part3, & the PIN must not appear anywhere in the program source.

~~Refuse~~ Reuse from previous Labs

Use EEPROM from part2 without changing its interface. PIN entry & display handling belong to the application, not the EEPROM routines. Bring back the Keyped scanner & DGL7137 display from Lab2. A PIN entry routine can look much like parseRx from the UART Lab: Recognize one key press event at a time, collect digits in RAM, & track how many digits have been entered.

KeyPad Behavior Table 3-1

Key	Action
0 – 9	Add one digit to the PIN entry in RAM
*	Cancel the entry and clear the collected digits
#	Submit the entry for comparison with the stored PIN
A – D	No PIN-entry action

- Accept one digit per debounced physical key press, not one per keypad scan.
- Accept exactly four digits followed by #.
- A # press with fewer than four digits rejects & clears the entry. Reject any entry if a fifth digit is pressed; do not accept the first four digits as a valid PIN.
- After a correct entry, drive the simulated solenoid output High for approximatly one second, then Low. An incorrect or incomplete entry must not activate it.
- Clear the collected entry after #, after *, & after reset. A user can then try again.
- Provide useful one character Progress/status feedBack on the DGL7137 display. Choose the characters & document their meanings in the LabBook.

Stored PIN & startup

Use EEPROMRead to retreive the four digit PIN. On startup or reset, keep the solenoid off.

Key Press feedBack

Use an active buzzer that produces its own tone when powered. Sound the buzzer for 100-250ms on every press of any 16 digit key. This feedback is on button presses only & does Not indicate a correct digit.

ART
10/22/26

Experiment 9 EEPROM

Part 3 - Four-Digit Electronic Lock

Goal

Extend Part2 into a Keypad-Controlled electronic Lock. Enter a Four-digit PIN followed by # to activate a simulated Solenoid output for one Second, The PIN is read from data EEPROM.

Saftey: Do not connect a Solenoid directly to a PIC output. A real inductive Load requires an appropriate driver & transient protection. Keep the simulated output inactive during reset & startup as well as normal locked operation.

Initial Pin Setup

Before running Part3, manually program a four-digit PIN into four consecutive data EEPROM addresses, with one decimal digit (0-9) in each address. Choose the addresses & varify the stored digits. This is a one-time setup. PIN changes are not part of Part3, & the PIN must not appear anywhere in the program source.

~~Refuse~~ Reuse from previous Labs

Use EEPROM from part2 without changing its interface. PIN entry & display handling belong to the application, not the EEPROM routines. Bring back the Keyped scanner & DGL7137 display from Lab2. A PIN entry routine can look much like parseRx from the UART Lab: Recognize one key press event at a time, collect digits in RAM, & track how many digits have been entered.

KeyPad Behavior Table 3-1

Key	Action
0 – 9	Add one digit to the PIN entry in RAM
*	Cancel the entry and clear the collected digits
#	Submit the entry for comparison with the stored PIN
A – D	No PIN-entry action

- Accept one digit per debounced physical key press, not one per keypad scan.
- Accept exactly four digits followed by #.
- A # press with fewer than four digits rejects & clears the entry. Reject any entry if a fifth digit is pressed; do not accept the first four digits as a valid PIN.
- After a correct entry, drive the simulated solenoid output High for approximately one second, then Low. An incorrect or incomplete entry must not activate it.
- Clear the collected entry after #, after *, & after reset. A user can then try again.
- Provide useful one character Progress/status feedBack on the DGL7137 display. Choose the characters & document their meanings in the LabBook.

Stored PIN & startup

Use EEPROMRead to retreive the four digit PIN. On startup or reset, Keep the solenoid off.

Key press feedBack

Use an active buzzer that produces its own tone when powered. Sound the buzzer for 100-250ms on every press of any 16 digit key. This feedback is on button presses only & does Not indicate a correct digit.

ART
10/22/26

Before Lab

Prepare:

- The Keypad Scanner, display wiring, & debouncing from Lab 2. (pg 155)
- electrical Simultaneous Keypad presses precautions (pg 157)
- Buzzer & Solenoid wiring, Loading codes, & reset safe output behavior.
- Draw a state diagram covering startup, locked operation, PIN entry, Cancel, Comparison, & the one second output.
- flowcharts for key entry & PIN comparison
- Document how EEPROM Read is used to retrieve the PIN.

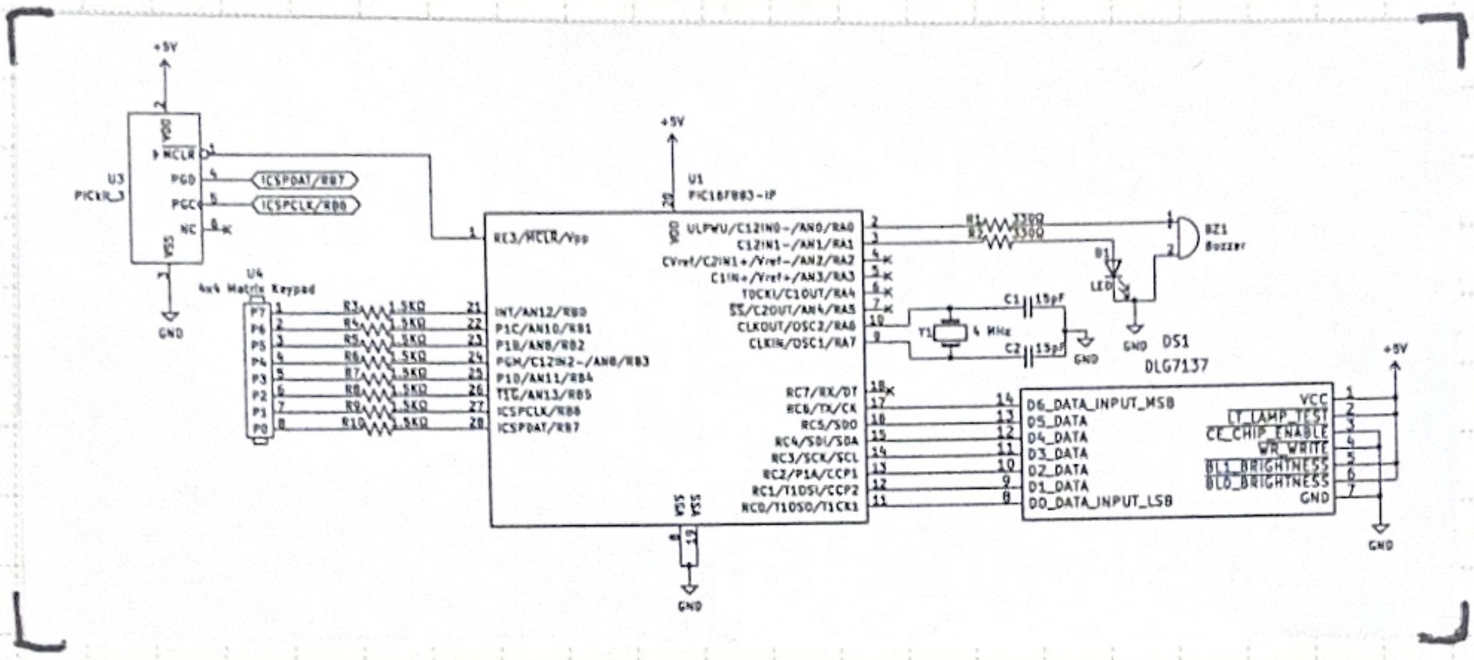


Figure 3-1 Solenoid & Buzzer standin circuit

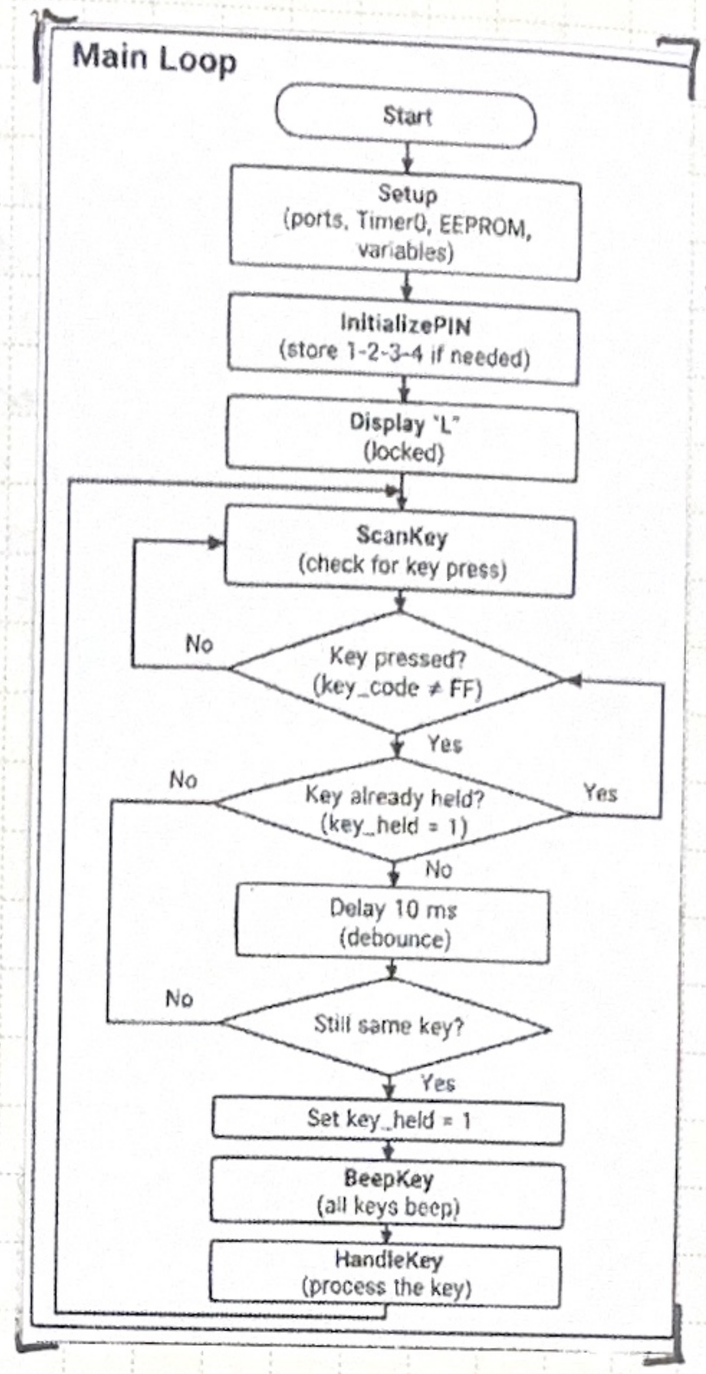
型 号 Type	TMB-12A01	TMB-12A03	TMB-12A05	TMB-12A12
额定电压(V) Rated Voltage	1.5	3	5	12
电压范围(V) Opcration Voltage	1.2～2.5	2.5～4	4～7	8～15
额定电流(mA) Rated Current	≤20	≤35	≤30	≤20
声压电平(dB) Sound Output	≥80	≥82	≥85	≥85
谐振频率(Hz) Resonant Freq	2300±500	2300±500	2300±500	2300±500
工作温度(℃) Operating Temp	-20～+60	-20～+60	-20～+60	-20～+60
储存温度(℃) Storage Temp	-20～+70	-20～+70	-30～+85	-30～+85
重 量(g) Weight	2	2	2	2

Table 3-2 Buzzer Specs

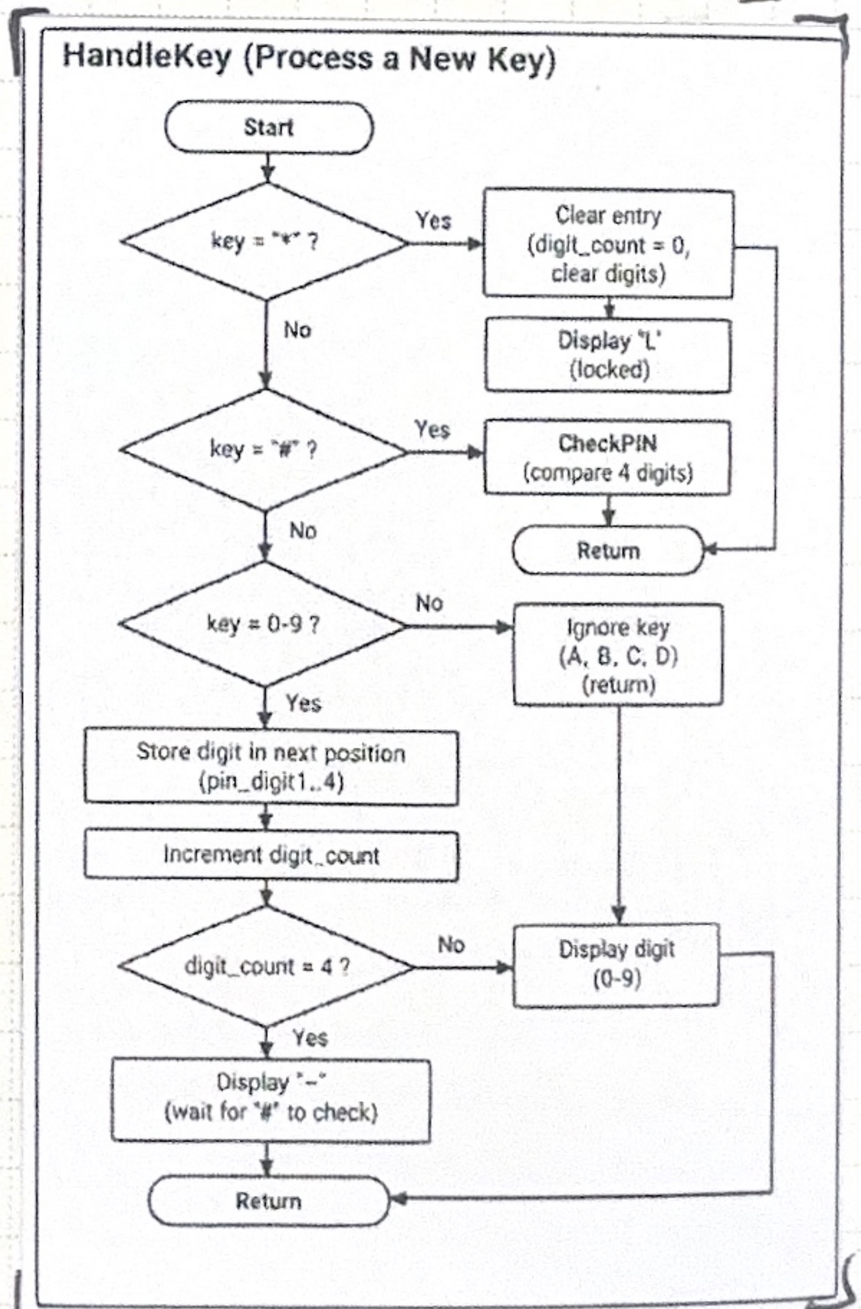
Based on observations, the current measured was 23mA and is out of specification. However, with the circuit using the TMB12A05 for only 150-250ms, allowing for the component to be used safely.

```
Main:
    ; Check for a keypad press
    CALL ScanKey
    BANKSEL key_code
    MOVLW NO_KEY
    XORWF key_code,W
    BTFSC STATUS,2
    goto KeyReleased

    ; Ignore a key that is still being held
    BTFSC key_held,0
    goto Main

    ; Debounce a new key press
    MOVF key_code,W
    MOVWF candidate_key
    CALL Delay10ms
    CALL ScanKey
    BANKSEL candidate_key
    MOVF candidate_key,W
    XORWF key_code,W
    BTFSS STATUS,2
    goto Main                          ; Not a stable press
    BSF key_held,0                     ; Count this press only once
    CALL BeepKey                       ; All 16 keys produce a beep
    CALL HandleKey                     ; Process the new key
    goto Main

; Handle one debounced keypad press
HandleKey:
    BANKSEL key_code
    MOVLW STAR_KEY
    XORWF key_code,W
    BTFSC STATUS,2
    goto CancelPIN
    MOVLW HASH_KEY
    XORWF key_code,W
    BTFSC STATUS,2
    goto SubmitPIN

    ; A-D do not change PIN entry
    MOVLW 0x0A
    SUBWF key_code,W                   ; Carry=1 when key >= 10
    BTFSC STATUS,0
    RETURN

    ; Ignore digits after a fifth digit was already rejected
    BTFSC entry_error,0
    RETURN

    ; Reject a fifth digit, even if first four were correct
    MOVLW PIN_LENGTH
    XORWF digit_count,W
    BTFSC STATUS,2
    goto TooManyDigits

    ; Store the entered digit: FSR = pin_digit1 + digit_count
    MOVF digit_count,W
    ADDLW pin_digit1
    MOVWF FSR
    MOVF key_code,W
    MOVWF INDF
    INCF digit_count,F

    ; Display how many digits have been entered (1-4)
    MOVF digit_count,W
    ADDLW 0x30                         ; ASCII '1' through '4'
    CALL DisplayChar
    RETURN
```

Code 3-1 mainLoop & Key handling



FC 3-1 Main Loop



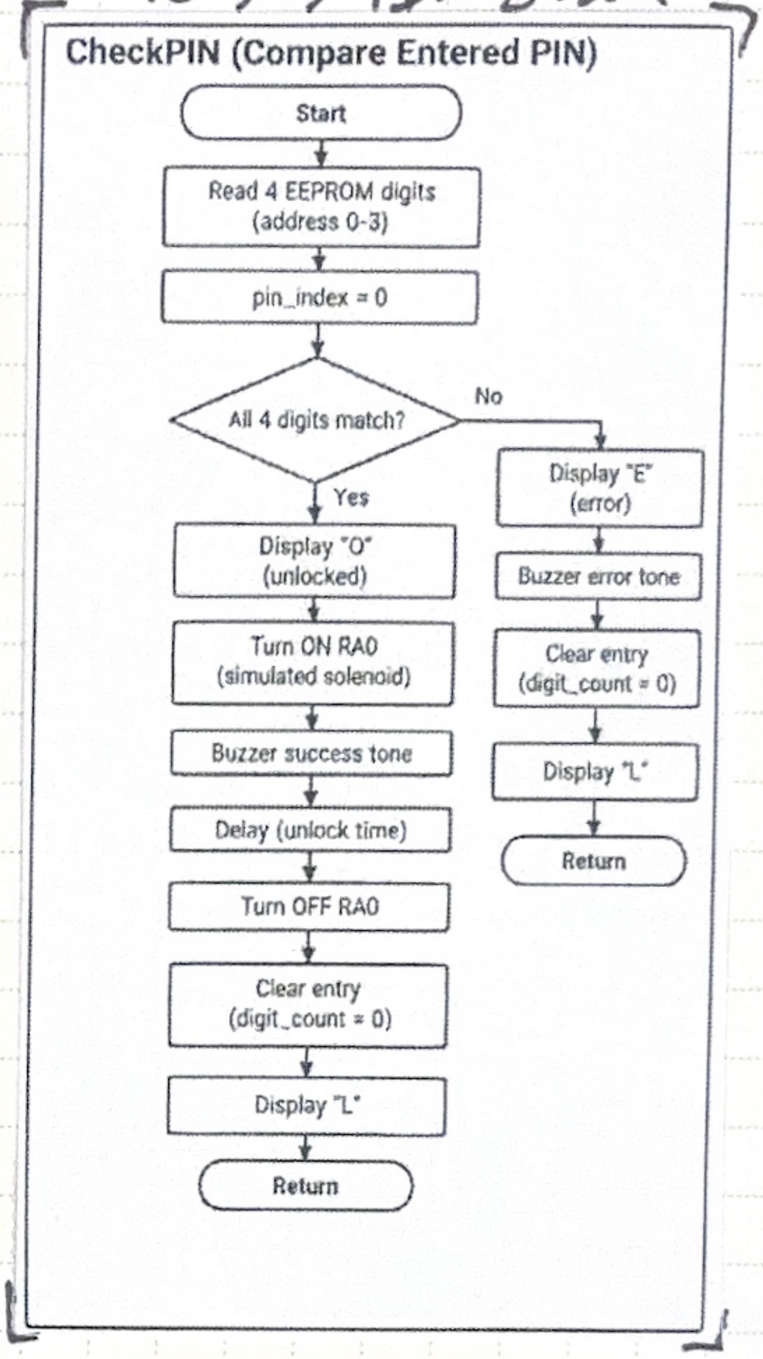
FC 3-2 Key handling

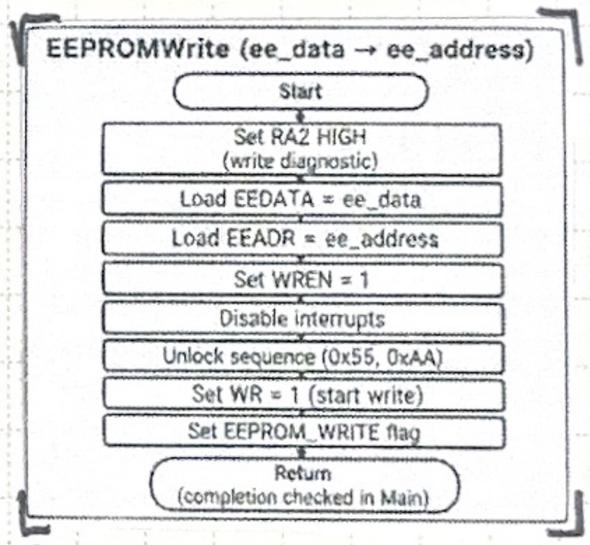
Experiment 9 EEPROM

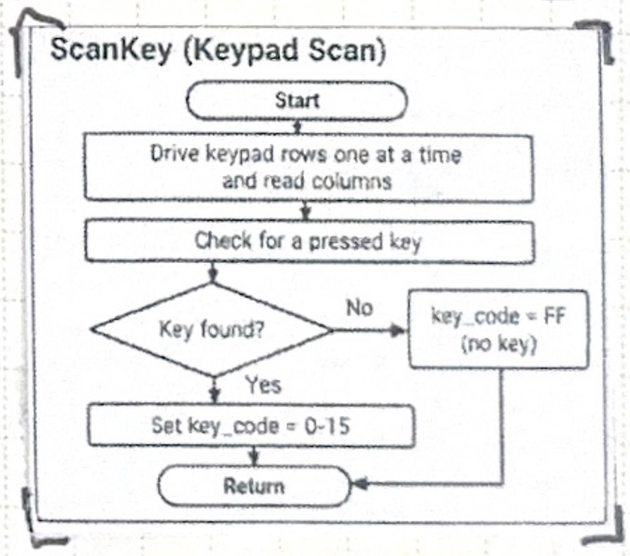
ART 10/26/26

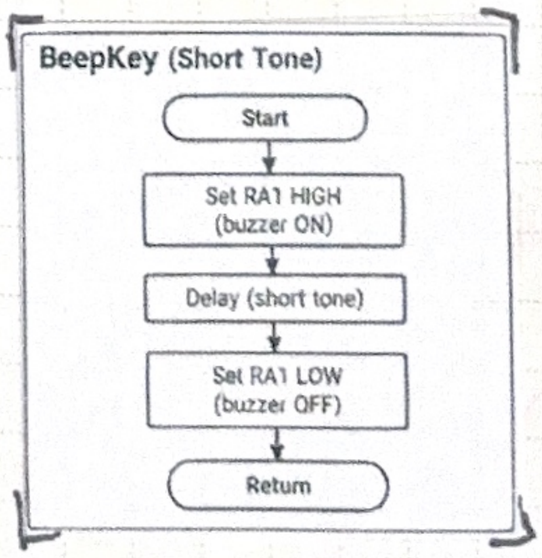
```
CompareNextDigit:
    MOVF pin_index,W
    ADDLW PIN_EE_START
    MOVWF ee_address
    CALL EEPROMRead              ; Part 2 routine, stored digit returns in ee_data

    BANKSEL pin_index            ; EEPROMRead changes banks
    MOVF pin_index,W
    ADDLW pin_digit1
    MOVWF FSR
    MOVF INDF,W                  ; Entered digit
    XORWF ee_data,W              ; Compare with stored EEPROM digit
    BTFSS STATUS,2
    goto WrongPIN
    INCF pin_index,F
    MOVLW PIN_LENGTH
    XORWF pin_index,W
    BTFSS STATUS,2
    goto CompareNextDigit

    ; All four digits matched
    MOVLW 0x55                   ; 'U' = unlocked
    CALL DisplayChar
    CALL SolenoidPulse
    MOVLW 0x4C                   ; 'L' = locked again
    CALL DisplayChar
    RETURN

WrongPIN:
    MOVLW 0x45                   ; 'E' = incorrect PIN
    CALL DisplayChar
    RETURN

TooManyDigits:
    BSF entry_error,0
    MOVLW 0x45                   ; 'E' = rejected entry
    CALL DisplayChar
    RETURN

CancelPIN:
    CALL ClearEntry
    MOVLW 0x43                   ; 'C' = canceled
    CALL DisplayChar
    RETURN

SubmitPIN:
    ; An invalid length or fifth digit must never unlock
    BTFSC entry_error,0
    goto RejectPIN
    MOVLW PIN_LENGTH
    XORWF digit_count,W
    BTFSS STATUS,2
    goto RejectPIN
    CALL ComparePIN
    CALL ClearEntry
    RETURN

RejectPIN:
    CALL ClearEntry
    MOVLW 0x45                   ; 'E' = wrong/incomplete PIN
    CALL DisplayChar
    RETURN

ClearEntry:
    BANKSEL digit_count
    CLRF digit_count
    CLRF entry_error
    CLRF pin_digit1
    CLRF pin_digit2
    CLRF pin_digit3
    CLRF pin_digit4
    RETURN
```

FC 3-3 PIN check









Code 7-2 Digit handeling for >5 or */# (Greater than 5)

Important Variables (GPRs)

pin_digit1..4	- entered PIN digits	ee_status	- status bits (READ, WRITE, ERROR)
digit_count	- number of digits entered (0-4)	entry_error	- 1 if fifth digit entered
key_code	- scanned key (0-15, FF = none)	pulse_count	- Timer0 overflow counter
key_held	- 1 when a key is being held	TIMEOUT_OVERFLOWS = 15	(~1 second)
ee_address	- EEPROM address (0-3)	PIN_EE_START = 0x00	
ee_data	- EEPROM read/write data	PIN_LENGTH = 4	

Keypad Codes

0 - 9	; numeric digits
A, B, C, D	; ignored
* (0x0A)	; clear entry
# (0x0B)	; check PIN
FF (0xFF)	; no key

EEPROM Status Bits (ee_status)

bit 0	EEPROM_READ	- 1 while a read is in progress
bit 1	EEPROM_WRITE	- 1 while a write is in progress
bit 2	EEPROM_ERROR	- 1 if write verification fails
bit 3	GIE_SAVED	- saved global interrupt enable

GPR's, Codes, & Status bits

ART 10/26/26

```
InitializePIN:
    CLRF EEPROMAddress
    MOVLW 0x01
    MOVWF EEPROMData
    CALL EEPROMWriteINIT

    MOVLW 0x01
    MOVWF EEPROMAddress
    MOVLW 0x02
    MOVWF EEPROMData
    CALL EEPROMWriteINIT

    MOVLW 0x02
    MOVWF EEPROMAddress
    MOVLW 0x03
    MOVWF EEPROMData
    CALL EEPROMWriteINIT

    MOVLW 0x03
    MOVWF EEPROMAddress
    MOVLW 0x04
    MOVWF EEPROMData
    CALL EEPROMWriteINIT

    RETURN

EEPROMWriteINIT:
    BANKSEL EEADR
    MOVF EEPROMAddress,W
    MOVWF EEADR                     ;the address to write
    MOVF EEPROMData,W
    MOVWF EEDATA                    ;the byte to write
    BANKSEL EECON1
    BCF EECON1,7                    ;pick data EEPROM
    BSF EECON1,2                    ;allow writing

    GieOff:
    BCF INTCON,7                    ;turn interrupts OFF
    BTFSC INTCON,7                  ;make sure they are really OFF
    goto GieOff

    MOVLW 0x55                      ;the chip needs this exact code, in this exact order,
    MOVWF EECON2                    ;with nothing in between, or it ignores the write
    MOVLW 0xAA
    MOVWF EECON2
    BSF EECON1,1                    ;start the write (we do not wait)

    BANKSEL PIR2
    BTFSS PIR2,4                    ;is EEIF still set
    goto $-1
    BCF PIR2,4                      ;Clear EEIF
    BANKSEL EECON1
    BCF EECON1,2
    RETURN
```

ART
10/26/26

* This code was flashed one time & was removed from the source code so no password was present in source code.

Code 3-3 Temporary EEPROM Write code to instanciate PIN values

FSR
10-26-26

ART
10/26/26

Part 4 - Authenticated PIN Change

Goal

Extend the Part3 lock so the user can set a PIN on first use & change it with the keypad. Store the pin & its validity in EEPROM.

PIN Storage & startup

Use five consecutive EEPROM addresses: four for the PIN digits (0-9) & one for a validity marker. Choose & document the addresses.

- Marker = 0xC4 : The stored PIN is valid
- Any other marker value: the PIN is invalid

Before the first Part4 Run, use the programmer to set the marker to an invalid value, such as 0xFF

Service Modes

At startup, the stored PIN-valid marker determines wheather the lock begins in invalid PIN mode or normal locked mode. The keypad must be serviced in all modes & must not be blocked for more than 20ms, except during the one-second solenoid activation & when a new PIN is written to EEPROM. The buzzer must sound for 100-250ms on every debounced keypress in all modes. Holding a key must produce only one key press; another press is accepted only after the key is released & pressed again. Each PIN entry accepts only digits 0-9, followed by # after exactly four digits. Use the DLG2137 to show whether the current or new PIN is being requested & whether saving succeeds or fails. choose & document the display characters.

Invalid PIN Mode

The solenoid never activates in invalid PIN mode.

1. Enter a new four-digit PIN followed by #.
2. Enter the same four digits followed by #.

Matching entries are saved as the new PIN. After a successful save, the lock enters normal locked mode. A mismatch, incomplete entry, unexpected key, or * clears the entry & restarts invalid PIN mode without saving.

Normal locked Mode

1. Enter the current four-digit PIN followed by # to unlock.
2. To change the PIN, press C followed by #.

A correct PIN activates the simulated solenoid for approximately one second. The lock remains in normal locked mode. An incorrect or incomplete entry, unexpected key, or ** * clears the entry & restarts normal locked mode. If C is not the first key or is not followed by #, return to normal locked mode.

PIN changed Mode

The solenoid never activates in PIN change mode.

1. Enter the current four-digit PIN followed by #.
2. Enter a new four-digit PIN followed by #. Enter the new PIN again followed by #.

If the current PIN is correct & the two new entries match, save the new PIN & return to normal locked Mode. An incorrect current PIN, mismatched new PINs, incomplete entry, unexpected key, or * cancels the change & returns to normal locked mode. While saving a new PIN, the lock ignores keypad input & keeps the solenoid off. EEPROM write fails use PIN Error behavior described next.

Experiment 9 EEPROM

AHT 10/12/26

Reusable PIN-Matching Routines

Consider two application subroutines. Both can use indirect addressing with FSR & ~~IDNF~~ INDF for PIN digits held in RAM.

- Current PIN Match: Collect the keypad sequence dddd#, where each d is a digit from 0-9. Consider four consecutive GPR locations with a pointer or index. Store each digit & advance the pointer. The fifth key press must be #; do not store it or advance the pointer. * or any unexpected key aborts. After #, use EEPROM Read to compare the four entered digits with the stored PIN. Report whether they match. Use this routine for normal unlocking & for authenticating a PIN change; the calling code decides whether to activate the solenoid.
- New PIN Match: Collect the keypad sequence dddd#dddd#. Consider eight consecutive GPR locations, four for each new PIN entry. Store each digit from 0-9 & advance the pointer. The fifth & tenth key presses must be #; do not store them or advance the pointer. * or any unexpected key aborts, After the final #, compare the two groups of four digits in RAM & report whether they match. This routine does not write EEPROM.

Shared helper routines may be useful. Use RAM GPRs as needed to track key entry, state, pointer or index values, match flags, & other working data. Document the GPR assignments & how each routine uses them in the lab book.

EEPROM PIN write Sequence (PINWrite:)

Create an application subroutine named PINWrite: to manage the six EEPROM write steps below. Use the Part 2 EEPROMWrite & EEPROMVerifyWrite routines without changing their interfaces. Start PINWrite only after New PINMatch succeeds &, for a change in normal locked mode, CurrentPINMatch has authenticated the current PIN. Set a RAM PIN_UPDATE flag before the first write. While it is set, keep the solenoid OFF & suspend normal lock operation. PINWrite handles EEPROM write completion & verification.

Table 4-1

Step	EEPROM location	Value to write
1	PIN-valid marker	0x00 (invalid)
2	PIN digit 1	First new digit
3	PIN digit 2	Second new digit
4	PIN digit 3	Third new digit
5	PIN digit 4	Fourth new digit
6	PIN-valid marker	0xC4 (valid)

Complete & verify each step before starting the next. check EEPROM_ERROR after verification. EEPROMWrite may skip a byte that already contains the requested value; Confirm that the byte is correct before advancing. During PINWrite, intentionally block all other CPU work. Wait for each EEPROM write to finish, verify it, & continue to the next step. Do not resume normal operation until the sequence completes or stops with an error.

- Success: After step 6 is verified, clear PIN_Update & return to normal locked operation with the new PIN
- Write or Verification error; Stop the sequence. Do Not write any remaining bytes. Keep the solenoid OFF, activate error indicator on portH & letter on C. Power cycle to try again
- Power Interruption: On the next startup, check the marker normally. If it is invalid, require two matching PIN entries before trying PINWrite again.

AHT 10/17

Experiment 9 EEPROM

ART
10/27/26

Before Lab

Update the part 3 state diagram to show invalid PIN mode, normal locked mode, PIN change mode, PINWrite, Success, & error handling. Include the CurrentPINMatch & NewPINMatch flows. Draw a flowchart for PINWrite showing all six EEPROM steps & the verification/error decisions. Document the five EEPROM addresses, PIN_UPDATE, display characters, & the PortA error indicator with its loading calculation.

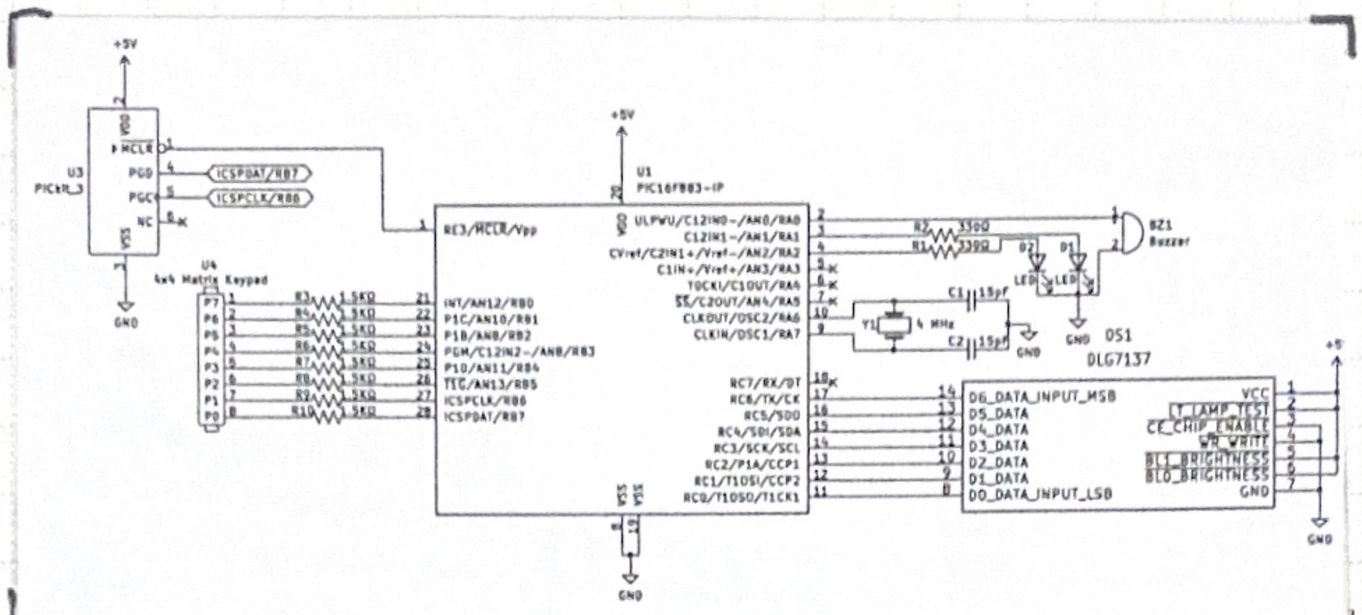


Figure 4-1
Added Error indicator light

Mode	Value	Meaning	Display prompt
MODE_INVALID_NEW	0	No valid PIN, enter a new one	N
MODE_INVALID_CONFIRM	1	No valid PIN, enter it again	F
MODE_LOCKED	2	Normal operation	L
MODE_ARMED	3	C pressed, waiting for #	C
MODE_CHANGE_AUTH	4	Enter the current PIN	A
MODE_CHANGE_NEW	5	Enter the new PIN	N
MODE_CHANGE_CONFIRM	6	Enter the new PIN again	F

Table 4-2 Mode display letter Meanings

The code structure for this part is the same as pt 3 with added EQU's to track mode position & more data slots for new pins & confirm pins. The confusion in this code is keeping track of how many digits have been pressed so it can properly throw errors if an unknown key is pressed in the middle of PIN changing. The program prompts for a new PIN on the first power up, it is saved permanently unless the password corrupts. The password can be changed by pressing C - # - d_1 - d_2 - d_3 - d_4 - # - d_1 - d_2 - d_3 - d_4 # - d_1 - d_2 - d_3 - d_4 - #

Original password new password repete new password

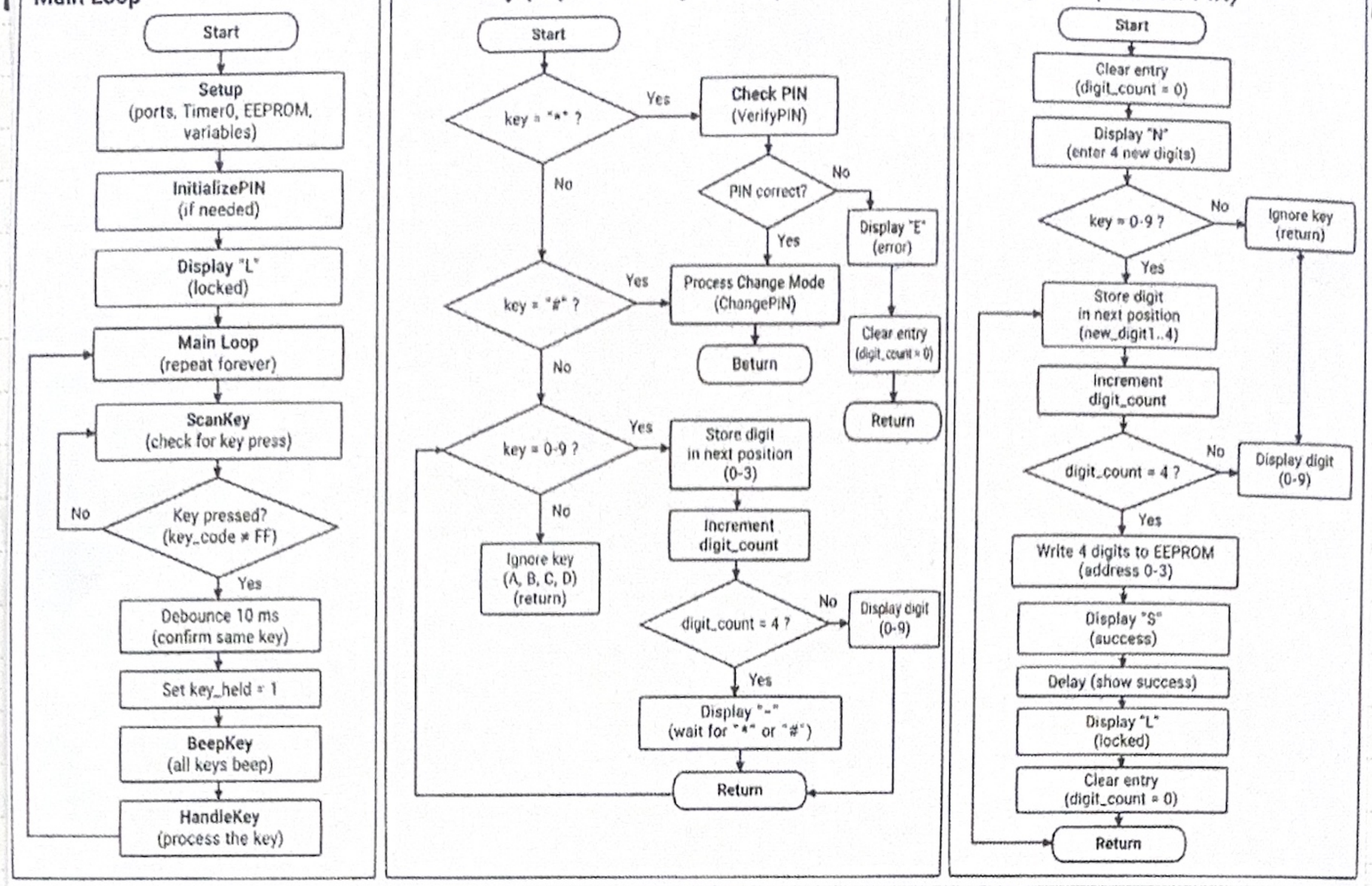
Interrupts are not being used although Timer1 is being used in the servicebeep routine that gets checked every main Loop.

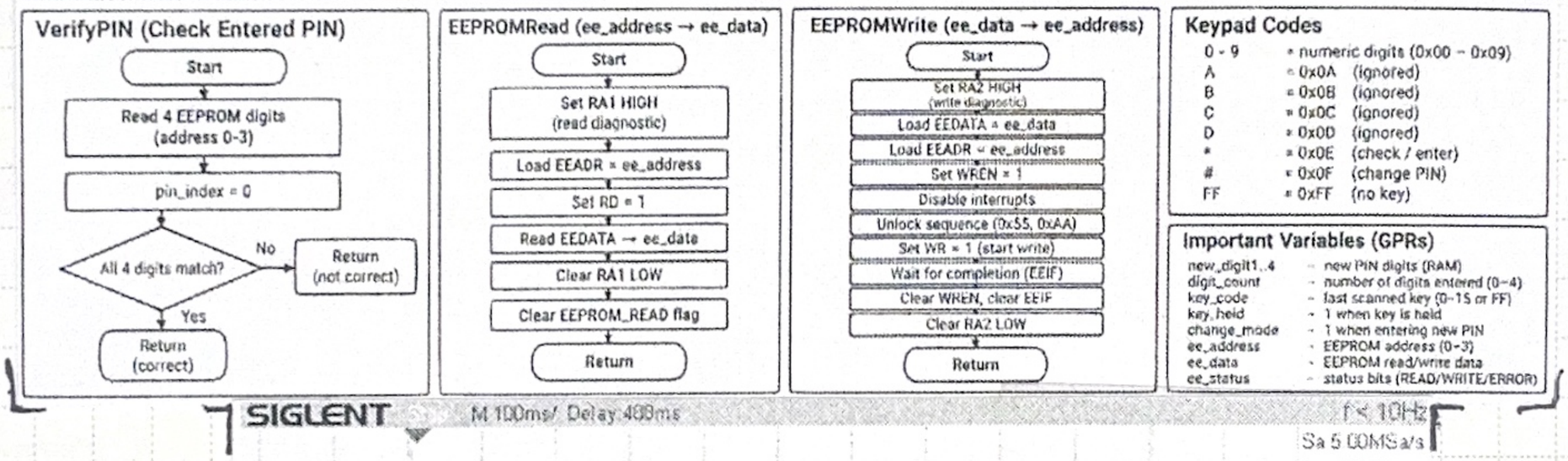
If an Error ever occurs, the program holds in Error mode until a power cycle is made.

Experiment 9 EEPROM

ART 10/28/26

FC 4-1 for program in Party





Unlocked Solenoid that Blocks all other operation →

Keypad being constantly Scanned →

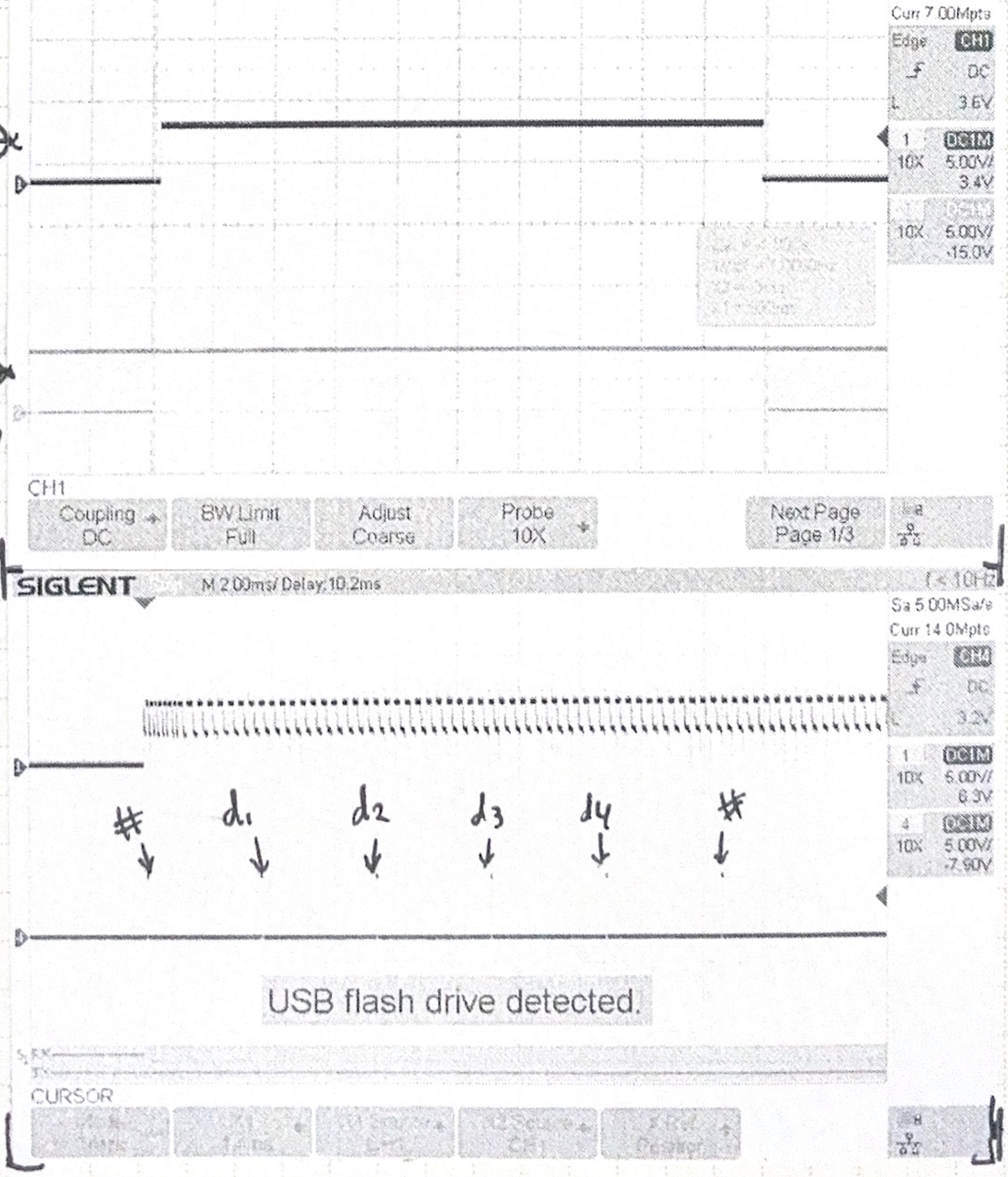


Figure 4-2 Solenoid unlock state Blocking

Figure 4-3 EEPROM writing new password

TJR 10-28-26

ART 10/29/26

Expiriment 9 EEPROM

ART
10/28/26

Conclusion

This expiriment centers around EEPROM (Electrically Erasable programmable ROM). Which is essentially GPR space on the pic that is nonvolitile, meaning it persists through power cycles. Any data on the EEPROM can only be changed if it is written to in a specific manner. The example that is most prevdent in this Lab is a 4x4 Keypad password. The password could be written to EEPROM Seperately from ~~they~~ the program structure so the password isnt clear in code. This remains true if you give the ability to use the Keypad to overwrite the saved PIN. This is done in a careful way to protect the program memory when writing to EEPROM.

ART
10/28/26